LateGraph: Forgetting and Repairing the Past in Temporal Graph Networks

Project milestone

Yijin Li
yjli88@stanford.edu

Lexi Zhou
lexi-zhou@stanford.edu

Where we are. Everything planned for the milestone is built and runs end to end in PyG, and most of what we planned for the final is done too: six datasets (including the 4.9M-event tgbl-review), three memory models, 3–5 seeds, and a membership attack with counterfactual shadow streams. Code and a Colab notebook: `https://github.com/YOUR-ACCOUNT/lategraph`.

- **Deleting from the log does not delete from the memory.** After a deletion request, a likelihood-ratio attack that observes the model's scores (knowing the weights and the other events) separates deleted from never-seen interactions with AUC 0.84 on Wikipedia, 0.86 on Reddit and 0.97 on tgbl-review, while the score alone gives 0.58 on Wikipedia. The leak is unchanged 50 steps after the request, and resetting or regenerating the memories does not remove it (§5.2).
- **Exact repair is exact.** Recomputing only the edit's dependency cone returns the memory of the corrected stream to within 1.0e-06 in float32 over 12,824 random edits on five datasets, bit for bit in float64, and at chance against every attack (§5.1).
- **Repair is fast for recent edits only.** For an edit that changes the memory, a step index makes repair 8.4× faster than replay on Wikipedia and 5.9× on Reddit; on hub-heavy MOOC and LastFM, and for old edits, the cone covers most later updates and replay is as fast (§5.1).
- **Cheap repair can be certified.** A memory cell that contracts for every input makes our error bound for approximate repair a theorem (it held in every trial), at a cost of 6.3 MRR points (§5.3).
- **Late events barely matter for accuracy.** Applying them in arrival order is within the seed spread of exact repair, also on SEC Form 4 with real delays and 687 real amendments; waiting with a buffer costs accuracy (§5.4).

1 The problem

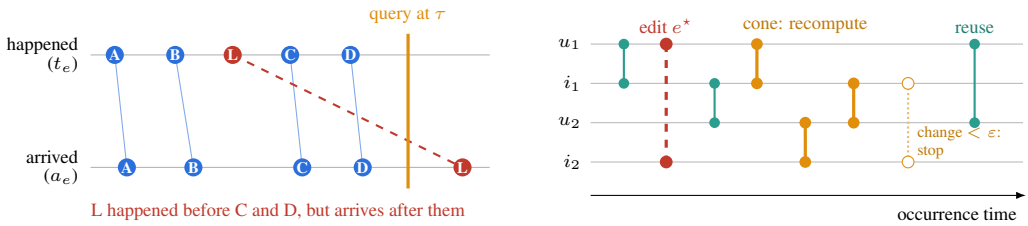
Streaming recommenders and transaction monitors use memory-based temporal GNNs such as TGN [4], JODIE [3] and DyRep [5]. They keep one memory vector per node and update it at every interaction, which is what makes them fast. Real event logs, however, change after the fact: an insider trade reaches the SEC days after it happened, a filing is amended (Form 4/A), a payment is reversed, a user asks for their data to be deleted. Each of these edits belongs *in the past*, before updates the model has already made. Because every update reads the memories it touches, one edit changes every later update that read a changed memory: the edit's *dependency cone* (Fig. 1). The GRU in the update cannot be inverted, so the cone has to be recomputed. Our target is the memory $S^\star(\tau)$ the model would hold at time τ had it processed every received event in the order it happened. After a deletion, $S^\star$ is the memory of a model that never saw the deleted events, so the question is also a privacy question: does deleting a row from the log make the model forget it?

2 What changed since the proposal

Done as planned: the retroactive memory, the time-ordered neighbour store, edit generators, the SEC graph, and repair against all baselines, including a StreamTGN-style rebuild [6] and a TIGER-style learned restart [7]. **Changed or added:**

- *The go/no-go test from the proposal feedback moved our emphasis to deletion.* Full replay is costly and buffering is not a cheap fix, but applying late events in arrival order loses almost

CS 224W (Fall 2026) project milestone. Project type 1: real-world application of GNNs.



(a) two clocks per event

(b) the dependency cone of one edit

Figure 1: (a) Every event has an occurrence time and an arrival time. At query time τ the late event L has not arrived, and when it does, C and D have already been applied without it. (b) Lines are nodes; vertical links are events, each updating both endpoints. An edit $e^\star$ (late insertion, deletion or amendment) changes every later update that reads a changed memory (orange). Exact repair recomputes only these, in time order, and copies everything else from a journal of saved memories (green); ε-repair stops where the change becomes small.

nothing, so for late events alone there is little accuracy to win back. The weight moved to edits that must be honoured exactly, above all deletions, where we found the memory leak.

- *A stronger, calibrated attack.* Besides the model's own score and a probe on the memories, a likelihood-ratio attack (LiRA [1]) with counterfactual shadow streams, which we also weaken (partial knowledge of the other events, later attacks, fewer candidate pairs) to see how strong an attacker must be.
- *More models and data:* DyRep (whose update reads the partner's neighbourhood), JODIE, LastFM and tgbl-review from TGB [2]; 5 seeds on Wikipedia and Reddit, 3 elsewhere.
- *A step index* so that repair visits only the steps the cone touches, and an honest account of when it pays off (recent edits) and when replay is as fast (old edits on hub-heavy graphs).
- *A rigorous certificate.* The contraction penalty of the proposal only bounds measured Jacobians; a memory cell that contracts by construction makes the error bound hold for every input and enables δ-certified repair.
- *Deletions of training-period events* (where the weights saw the data too), compared with retraining.
- *Exact counterfactual explanations:* the effect of deleting any past event on a prediction, compared with attention weights.
- *Not done:* GraphMixer (our memory-free control is a one-layer TGAT-style model) and shadow-model attacks on the weights.

3 Data and processing

Table 1: Datasets (chronological splits).

	events	nodes	test events	edge features	days
Wikipedia	157,474	9,227	23,621	172	31
Reddit	672,447	10,984	100,867	172	31
MOOC	411,749	7,144	61,763	4	30
LastFM	1,293,103	1,980	193,966	2	1,587
tgbl-review	4,873,540	352,637	728,919	1	7,053
SEC Form 4	1,048,386	69,536	156,943	25	1,003

JODIE and TGB. We load Wikipedia, Reddit, MOOC and LastFM with PyG's `JODIEDataset` and split by time 70/15/15, as `TemporalData.train_val_test_split`; tgbl-review comes from the TGB mirror on Hugging Face with TGB's own split. These datasets have one clock, so test events get arrival times from five delay models with a known ground truth: *random* (20% of events late, log-normal delay with a 2-hour median), *active users*, *outage bursts*, *offline users* and a *slow item shard* (Appendix A). The test stream is processed in steps of 200 events (one day for tgbl-review); an event can be used from the first step after it arrives.

SEC Form 4. We use the point-in-time view of the ZipLime insider-trading data [8] (Apache-2.0, built from SEC EDGAR), pinned to one revision. A transaction is an edge from the reporting insider to the issuer; its occurrence time is the trade date and its arrival time the EDGAR acceptance time; edge features (25-dim) encode the transaction code, size, price, derivative flag, role and 10b5-1 plan.

A Form 4/A row supersedes an earlier row: at its arrival, the old version is deleted and the new one inserted (687 amendments in the test period). In the test period 93% of transactions arrive after the day they happened, the median delay is 2.9 days, the 99th percentile 97 days, and 26% are filed more than 4 days after the trade (the legal deadline is two business days). To our knowledge no public temporal graph benchmark has real arrival times; we will publish this one.

4 Model, repair, attacks and metrics

Model. PyG's TGN [4] as in `examples/tgn.py`: `TGNMemory` with identity messages and last-message aggregation, a `TransformerConv` embedding over each node's last 10 neighbours and an MLP decoder (size 100; batch 200; Adam 10^{-4}; 15 epochs). When event $e = (u, v, x_e, t_e)$ is processed, endpoint u is updated by

$$s_u \leftarrow F(s_u, s_v, e, \tau_u) = \mathrm{GRU}\big([\, s_u \,\|\, s_v \,\|\, x_e \,\|\, \phi(t_e - \tau_u)\,],\ s_u\big), \tag{1}$$

where τ_u is u's previous update time and ϕ a learned time encoding; within a step every update reads the memory from before the step. JODIE uses an RNN memory and a time projection; DyRep's update also attends over the partner's neighbours, so it reads their memories too.

Exact repair. `RetroactiveTGNMemory` subclasses `TGNMemory`, loads the trained weights and keeps a *journal*: for every update the node, the event, the partner and the memory before and after. After an edit at step k_0 it walks forward and recomputes an update if its event changed or if it reads a memory that differs from the journal (the cone); every other result is copied. Updates outside the cone see identical inputs, so this returns exactly $S^\star$, provided the update is batch-invariant: BLAS uses another kernel for single-row products, which in float32 changes the time encoding of multi-day gaps by up to 0.06, so we pad single-row updates (Appendix B). A *step index* (the steps where each node is updated) lets repair visit only the steps that read a changed memory.

```
mem = RetroactiveTGNMemory.from_trained(tgn.memory, events)   # same weights as the trained TGNMemory
mem.warmup(0, stream.test0)                                    # history, in order
mem.advance(step_events)                                       # one step of the stream (journaled)
mem.edit(insert=late_events, delete=deleted_events)            # change the past (amendment = both)
stats = mem.repair("exact")                                    # or "eps" (eps=0.05), "replay", "khop"
z, last_update = mem(n_id)                                     # as TGNMemory in eval mode
```

Cheaper repair with a bound. *ε-repair* keeps the old memory where the recomputed one differs by at most $\varepsilon \, \|s^{\mathrm{old}}\|$. If every update shrinks memory differences by a factor $\rho < 1$, the error stays below $r/(1-\rho)$, and a per-node bound $b_n \leftarrow \rho \max(b_n, b_o) + \delta_n$ can be tracked online. A trained GRU is not contractive, so we compare a contraction penalty in training with a cell that contracts by construction, $s' = (1-a) \odot s + a \odot \tanh(W[s\|p\|x\|\phi] + b)$ with every row of the memory block of W at ℓ_1 norm at most c and $a \geq a_{\min}$, so that $\rho = 1 - a_{\min}(1-c)$ for every input; *δ-certified repair* skips a change only while the bound stays below δ.

Attacks. We take user–item pairs that first meet in the test period: half (A) are observed and deleted at a request step, half (B) never happen. The weights were trained before the test period, so anything that separates A from B lives in the memory. *Query access*: the model's link score, alone and in LiRA with 64 counterfactual shadow streams (random halves of the candidates observed and then deleted with the method under test; Gaussians with per-pair means). *White-box access*: a probe on the memories and LiRA on their cosine. AUC 0.5 means the deleted pairs look like pairs that never happened.

Protocol and metrics. Before step j is scored, every policy turns the events it has received into a memory; policies share the weights, the negatives and a neighbour store ordered by occurrence time. *Accuracy*: MRR of the true destination against 100 random and 100 historical negatives. *Cost*: node updates as a multiple of normal in-order processing, and wall-clock time on a laptop CPU. *Agreement*: top-1 predictions equal to exact repair. *Privacy*: membership AUC.

5 Results so far

5.1 Exact repair: exact, and fast for recent edits

Table 2: over 12,824 random insertions, deletions and amendments, the repaired memory equals a fresh sequential recomputation to within 1.0e-06 in float32 and bit for bit in float64; after a deletion

Table 2: Exact repair on random edits of a 100-step test window (insertions, deletions and amendments; 10% of the trials edit 2–10 events at once). Max. error: largest |repaired − reference| over all memory entries, where the reference is a sequential recomputation of the corrected stream (float32 unless noted). Single edits: share that changes no memory, and for the others the cone as a share of all updates since the edit (median and 90th percentile; *DyRep: median over all trials). Speed-up: median, over single edits that change a memory, of the time of rollback + replay over the time of exact repair with the step index (in brackets: walking every step), on an idle laptop CPU; edits that change nothing are recognised at their own step.

				single edits			
	trials	max. error	bit-identical	no change	cone	cone p90	speed-up
Wikipedia (TGN)	3,945	5.4e-07	98%	57%	0.48%	6.8%	8.4× (2.4×)
Reddit (TGN)	2,960	9.1e-07	72%	11%	0.38%	27.5%	5.9× (2.5×)
MOOC (TGN)	2,960	1.0e-06	70%	54%	52.16%	80.8%	0.9× (1.2×)
LastFM (TGN)	1,977	0	100%	26%	13.74%	62.8%	1.5× (1.6×)
tgbl-review (TGN)	982	6.0e-08	86%	6%	0.01%	0.04%	8.2× (1.8×)
Wikipedia (JODIE)	1,977	2.5e-06	49%	56%	0.52%	7.8%	–
Wikipedia (contractive)	1,977	3.0e-08	99%	56%	0.52%	7.8%	–
Wikipedia (DyRep)	1,500	1.6e-05	–	9%*	14.75%*	–	–
Wikipedia (TGN, float64)	983	0	100%	–	–	–	–

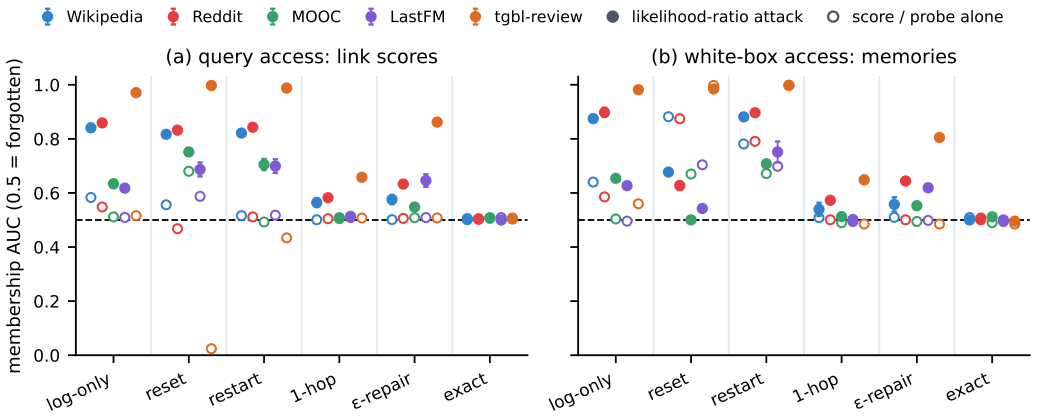


Figure 2: Membership AUC after a deletion request (0.5: deleted pairs look like pairs that never happened). (a) Query access: the model's link score alone (hollow) and in the likelihood-ratio test (filled). (b) White-box access: a probe on the two memories (hollow) and the likelihood-ratio test on their cosine (filled). Mean ± std over seeds.

the journal, too, is that of a memory that never saw the events (unit test). 57% of single edits on Wikipedia change no memory, because the event is not the latest of its nodes in its step. Among the others the cone is small on Wikipedia and Reddit but large on hub-heavy graphs (median 52.2% of the later updates on MOOC), and it grows with the age of the edit. For edits that change a memory, repair with the step index is 8.4× faster than replay on Wikipedia, 5.9× on Reddit and 8.2× on tgbl-review, but 1.5× on LastFM and 0.9× on MOOC; on windows of up to 500 steps, edits 200 or more steps old are repaired no faster than they are replayed (Fig. 3 in the appendix). Repair is the tool for recent edits, replay for old ones; both are exact. DyRep's cones are far larger (median over all edits 14.7% of the later updates on Wikipedia, against 0.02% for TGN), because its update reads whole neighbourhoods.

5.2 Deleting from the log is not enough

With query access alone, log-only deletion is exposed (Fig. 2a): the score by itself barely separates A from B (AUC 0.58 on Wikipedia), but LiRA on the same score reaches 0.84 on Wikipedia, 0.86 on Reddit and 0.97 on tgbl-review, and the memories make it 0.87, 0.90 and 0.98 (b). On MOOC the score is at chance even before any deletion, yet LiRA reaches 0.63. Resetting the memories and the

Table 3: The leak under other query-access attackers: LiRA AUC on the link score after log-only deletion (del.) and after resetting the memories (reset), mean over seeds. The attacker's shadow streams leave out the other test-period events it does not know; or it attacks 20 or 50 steps after the request; or fewer candidate pairs are unknown at once (K; at most 1,000 per group by default, 338 on Wikipedia). With partial knowledge and the default K, ε-repair, the 1-hop rebuild and exact repair stay at or below 0.52; the other rows expose them (Table 8). With $K = 25$ the AUCs are noisy: exact repair, which leaks nothing, ranges from 0.44 to 0.60.

attacker	Wikipedia		Reddit		MOOC		LastFM		tgbl-review	
	del.	reset	del.	reset	del.	reset	del.	reset	del.	reset
full knowledge, at the request	0.84	0.82	0.86	0.83	0.63	0.75	0.62	0.69	0.97	1.00
knows 50% of the other events	0.70	0.74	0.65	0.77	0.53	0.73	0.51	0.66	0.57	0.97
knows 10%	0.65	0.67	0.61	0.73	0.50	0.72	0.51	0.64	0.53	0.97
knows none of them	0.63	0.63	0.59	0.73	0.50	0.70	0.52	0.63	0.52	0.97
20 steps after the request	0.84	0.81	0.87	0.85	0.65	0.72	0.65	0.67	0.97	1.00
50 steps after	0.83	0.82	0.87	0.86	0.65	0.72	0.66	0.68	0.97	1.00
$K = 100$ candidate pairs	0.92	0.89	0.97	0.95	0.75	0.97	0.85	0.94	–	–
$K = 25$	0.97	0.95	0.98	0.98	0.84	1.00	0.91	0.99	–	–
$K = 25$, knows 50%	0.67	0.78	0.61	0.91	–	–	–	–	–	–
$K = 25$, knows none	0.63	0.65	0.59	0.80	–	–	–	–	–	–

learned restart fool the score but not LiRA (0.82 and 0.82 on Wikipedia), and the probe spots their regenerated memories (0.88 and 0.78). The 1-hop rebuild costs about as much as exact repair and, like ε-repair, leaves a small residue (0.56 and 0.58). Exact repair is at chance by construction, at 0.30× the normal work up to the request on Wikipedia against 0.92× for replay. JODIE and DyRep leak in the same way (LiRA 0.88 and 0.75 on Wikipedia).

How strong must the attacker be? Table 3: an attacker who knows none of the other test-period events still detects log-only deletion on Wikipedia and Reddit (0.63 and 0.59) and resets everywhere (0.63–0.97); 50 steps after the request the leak is unchanged (0.83 on Wikipedia); with fewer candidate pairs unknown at once ($K = 25$) log-only deletion is exposed with AUC 0.97 on Wikipedia and 0.84 on MOOC. The residues of ε-repair and the 1-hop rebuild are visible only to an attacker who knows almost every other event.

Training-period deletions. When the weights saw the events too, we compare with retraining from scratch without them (Table 4 in the appendix). Before any deletion the deployed model separates these pairs from never-seen ones only weakly (link-score AUC 0.55 / 0.52 / 0.49 on Wikipedia / Reddit / MOOC); log-only deletion leaves 0.52 / 0.51 / 0.49, and repairing the memory with unchanged weights 0.50 / 0.50 / 0.49, level with retraining (0.52 / 0.50 / 0.49); the probe agrees, also for pairs from the last 20% of the training period. These attacks are weak here, so we claim only that no signal remains that they detect; a shadow-model attack on the weights is future work. The cone of one training-period pair covers 49% of the later history updates on Wikipedia, so recomputing the memory from scratch with fixed weights, one batched pass over the history (0.28 s on an idle machine; Reddit: 1.60 s), is faster than repairing it (0.48 s and 5.58 s, measured during the experiment runs); both are exact, and far cheaper than retraining (5 minutes on Wikipedia).

5.3 Cheaper repair with a guarantee

Standard TGNs are not contractive (local Jacobian norms sum to 2.04, median), so our bound does not apply to them. A contraction penalty makes 99.2% of the updates contract at $\rho = 0.85$ for 1.3 MRR points, but only at the states it measured. The contractive cell contracts for every input: its proven per-node bound held in all trials and was within 5.2× of the true worst error (median), and δ-certified repair with $\delta = 0.02$ recomputes 0.45× the updates of exact repair while keeping every memory within 0.02 of the exact one; the price is 6.3 MRR points at $\rho \leq 0.9$ on Wikipedia (Table 5 in the appendix).

5.4 Late events and SEC Form 4

Applying late events on arrival, out of order, changes MRR by +0.13%, -0.13% and +0.13% on Wikipedia, Reddit and MOOC relative to exact repair, within the spread over seeds; a one-hour buffer changes it by -4.9%, -1.5% and -11.3%, because it holds back every recent event, not only the late ones (all delay models: Table 7; all policies: Table 6, both in the appendix). On SEC Form 4,

with real delays and amendments, every policy is within 0.24% of exact repair except dropping late events (-0.91%), while seeing every event on time would gain +1.7%: what lateness costs is the information that has not arrived, which no repair recovers. Exact repair is not free in wall-clock time (4.9 ms per step of 200 Wikipedia events against 0.7 ms in arrival order and 9.5 ms for replay), so for late events repair is a matter of correctness, not of accuracy.

5.5 Exact counterfactual explanations

Deleting each of a user's last 20 events in turn and repairing gives the exact effect of every event on a prediction (up to 200 predictions per model, 3 seeds; Fig. 4 in the appendix). About half of the influence (48% on Wikipedia) flows through the memory, and 9% comes from events outside the 10 the attention layer sees. The most-attended event is the most influential one for the neighbour-list channel in 45% of the predictions on Wikipedia (chance 10%), but for the total effect only in 16% (10% on MOOC): attention only partly explains what it weighs, and not the prediction.

6 Next steps (until December 10)

- **Nov 11–24** (Lexi): the Medium blog post (motivation, the cone animation, PyG snippets, the deletion attack, results), reviewed by the course staff before publishing; the Colab notebook as its companion (already runs end to end with saved outputs).
- **Nov 11–24** (Yijin): a shadow-model attack on the weights for training-period deletions; GraphMixer or DyGFormer as a stronger memory-free baseline; release of the SEC two-clock graph.
- **Nov 25–Dec 10** (both): figures and text polish, the public GitHub repository with every job list, and an optional GPU version of the repair loop.

7 Questions we would like feedback on

1. Is the privacy finding (log-only deletion leaks through the memory; resets leave a fingerprint) the right main story for the blog post, with exact repair as the fix and late events as a secondary result?
2. Our strongest attacker knows the weights and the other events; weaker ones still find log-only deletion on Wikipedia and Reddit. Which attacker would you consider realistic enough to put first?
3. Exact repair beats replay only for recent edits. Is it acceptable to recommend replay for old edits (both are exact), or should we invest in a faster (GPU) repair before the final?
4. On SEC most insiders trade a single stock, so link prediction is easy (MRR of exact repair 0.65). Would a node-level task be a better showcase for the real two-clock data?

Code. `https://github.com/YOUR-ACCOUNT/lategraph`: the `lategraph` package, 18 unit tests (our kernel equals PyG's `TGNMemory`; exact repair equals recomputation in float32 and float64; the step index equals the full walk bit for bit; the journal is purged), all experiment scripts with the job lists behind every number in this report, the JSON results, and a Colab notebook with saved outputs that trains a TGN, repairs a late event, runs the streaming evaluation, the deletion attack and the contractive cell, and redraws every result in about 20 minutes on a CPU.

References

[1] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr. Membership inference attacks from first principles. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.

[2] S. Huang, F. Poursafaei, J. Danovitch, M. Fey, W. Hu, E. Rossi, J. Leskovec, M. Bronstein, G. Rabusseau, and R. Rabbany. Temporal graph benchmark for machine learning on temporal graphs. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[3] S. Kumar, X. Zhang, and J. Leskovec. Predicting dynamic embedding trajectory in temporal interaction networks. In *KDD*, 2019.

[4] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein. Temporal graph networks for deep learning on dynamic graphs. In *ICML Workshop on Graph Representation Learning and Beyond*, 2020. arXiv:2006.10637.

[5] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha. DyRep: Learning representations over dynamic graphs. In *ICLR*, 2019.

[6] L. Zhang, P. Qiao, Z. Zhang, Y. Yuan, and G. Wang. StreamTGN: A GPU-efficient serving system for streaming temporal graph neural networks. arXiv:2603.21090, 2026.

[7] Y. Zhang, Y. Xiong, Y. Liao, Y. Sun, Y. Jin, X. Zheng, and Y. Zhu. TIGER: Temporal interaction graph embedding with restarts. In *WWW*, 2023.

[8] ZipLime. Insider trading (SEC Form 4, point-in-time). Hugging Face dataset, https://huggingface.co/datasets/ZipLime/insider-trading, 2026. Apache-2.0; source: SEC EDGAR.

A Training and evaluation details

Delay models. *Random*: 20% of the test events are late, with a log-normal delay of median 2 hours ($\sigma = 1$; tgbl-review: median 2 days); *active*: the 5% most active users send half of their events late; *burst*: 4 outages of 3 hours, during each of which the events of a random half of the users are held; *offline*: 20% of the users upload their events every 6 hours; *item shard*: 20% of the items live on a slow shard and half of their events are late.

Attacks. Deletion pairs: user–item pairs whose first interaction lies in the first half of the test period and whose user has at least three other events, at most 1,000 per group (338 on Wikipedia); the request is at the middle of the test period. LiRA: 64 shadow streams (16 for DyRep and tgbl-review). Weaker attackers: with knowledge f, a fixed random $1 - f$ of the other test-period events is left out of every shadow stream; with a delay of k steps, target and shadows process k more steps before the attack; with $K < 1{,}000$, fewer candidate pairs are drawn. The probe is a logistic regression on memory features trained with 5-fold cross-validation on the target's own labels.

Models. Memory, time and embedding size 100, 10 neighbours, 2-head `TransformerConv`, batch 200, Adam 10^{-4}, one random negative per positive, best epoch by validation AP (15 epochs; SEC 10; tgbl-review 6). The contraction penalty uses 64 sampled updates per batch and $\lambda = 10$; the contractive cell uses $c = 0.5$ and $a_{\min} = (1 - \rho)/(1 - c)$.

B Why float32 broke our first exactness test

The time encoding is $\phi(\Delta t) = \cos(\omega \Delta t + b)$ with trained $|\omega|$ up to about 1 and Δt in seconds, so $\omega \Delta t$ reaches 10^6 for a gap of about 12 days, where adjacent float32 numbers are 0.0625 apart. `torch.nn.Linear` uses a matrix–vector kernel when the batch has one row and a matrix–matrix kernel otherwise; the two round $\omega \Delta t + b$ differently, and the cosine turns a one-ulp difference into up to 0.06. Exact repair recomputes small subsets of a step while the reference recomputes whole steps, so without padding 20% of the trials disagreed by more than 10^{-5}. Padding single-row updates to two rows removes these errors; the remaining differences come from the summation order of batched kernels, and in float64 every repaired memory was bit-identical to the reference.

C More results

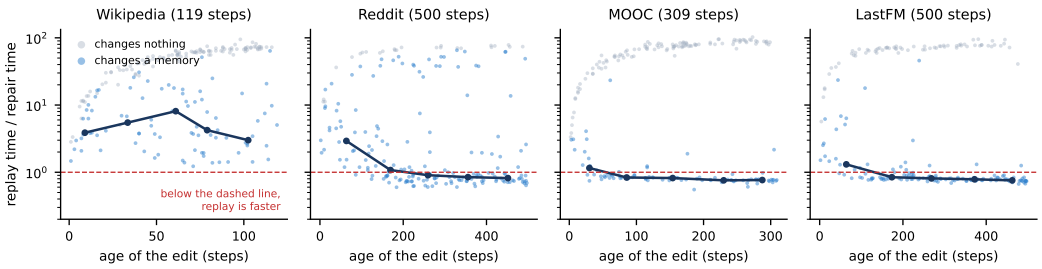


Figure 3: Time of rollback + replay over the time of exact repair (step index) for single edits at random positions of the longest available test window, against the age of the edit; line: median over edits that change a memory.

Table 4: Deleting training-period pairs (membership AUC of the link score / the memory probe at the deployment start; mean ± std over seeds, number of seeds in brackets). "Recent": pairs from the last 20% of the training period. Last row: time of one exact memory repair for a single pair / of recomputing the memory over the whole history with fixed weights (one batched pass, idle machine) / of retraining one model.

	Wikipedia (5)		Reddit (3)		MOOC (3)		Wikipedia, recent (3)	
	score	probe	score	probe	score	probe	score	probe
no deletion	0.55 ±0.02	0.52 ±0.01	0.52 ±0.01	0.51 ±0.01	0.49 ±0.03	0.52 ±0.01	0.58 ±0.02	0.54 ±0.02
log-only	0.52 ±0.02	0.52 ±0.02	0.51 ±0.01	0.51 ±0.01	0.49 ±0.03	0.52 ±0.02	0.54 ±0.01	0.54 ±0.02
memory repair	0.50 ±0.02	0.51 ±0.01	0.50 ±0.01	0.50 ±0.01	0.49 ±0.03	0.52 ±0.02	0.50 ±0.01	0.50 ±0.03
memory repair + fine-tune	0.49 ±0.00	0.50 ±0.03	0.49 ±0.01	0.51 ±0.05	0.49 ±0.03	0.48 ±0.02	0.51 ±0.02	0.48 ±0.03
retrain (reference)	0.52 ±0.02	0.51 ±0.02	0.50 ±0.02	0.50 ±0.01	0.49 ±0.02	0.50 ±0.03	0.49 ±0.02	0.47 ±0.02
cost: repair / recompute / retrain	0.48 s / 0.28 s / 5 min		5.58 s / 1.60 s / 32 min		0.82 s / 0.53 s / 7 min		0.08 s / 0.28 s / 4 min	

Table 5: Contraction and the error bound on Wikipedia ($\varepsilon = 0.05$; 100 random deletions per model and seed). Certified: updates with $\|J_{\text{own}}\| + \|J_{\text{partner}}\| < 1$ (all updates for the contractive cell, which contracts by construction). Bound held: share of trials in which the per-node bound held for every node (first order for the GRU, proven for the cell). δ-cert.: updates recomputed by δ-certified repair ($\delta = 0.02$) relative to exact repair.

	val. AP	test MRR	certified (%)	bound held (%)	bound / error	δ-cert. cost
standard TGN (GRU)	0.965 ±0.002	0.643 ±0.018	0.0 ±0.0	100	26.7	–
penalty, $\rho = 0.95$	0.956 ±0.001	0.656 ±0.005	20.3 ±1.6	100	1.2	–
penalty, $\rho = 0.85$	0.951 ±0.000	0.630 ±0.007	99.2 ±0.1	100	1.1	–
penalty, $\rho = 0.75$	0.946 ±0.001	0.619 ±0.006	99.9 ±0.0	100	1.0	–
penalty, $\rho = 0.85$, $\lambda = 1$	0.952 ±0.001	0.630 ±0.007	81.7 ±3.3	100	1.1	–
penalty, $\rho = 0.85$, $\lambda = 100$	0.948 ±0.000	0.621 ±0.002	99.8 ±0.0	100	1.1	–
contractive cell, $\rho \leq 0.95$	0.939 ±0.001	0.582 ±0.010	100	100 (proven)	5.1	0.46
contractive cell, $\rho \leq 0.9$	0.939 ±0.002	0.580 ±0.016	100	100 (proven)	5.2	0.45
contractive cell, $\rho \leq 0.8$	0.939 ±0.002	0.583 ±0.012	100	100 (proven)	5.2	0.44
contractive cell, $\rho \leq 0.95$, $c = 0.7$	0.942 ±0.002	0.587 ±0.016	100	100 (proven)	5.6	0.47
contractive cell, $\rho \leq 0.99$, $c = 0.9$	0.946 ±0.001	0.606 ±0.008	100	100 (proven)	7.0	0.50

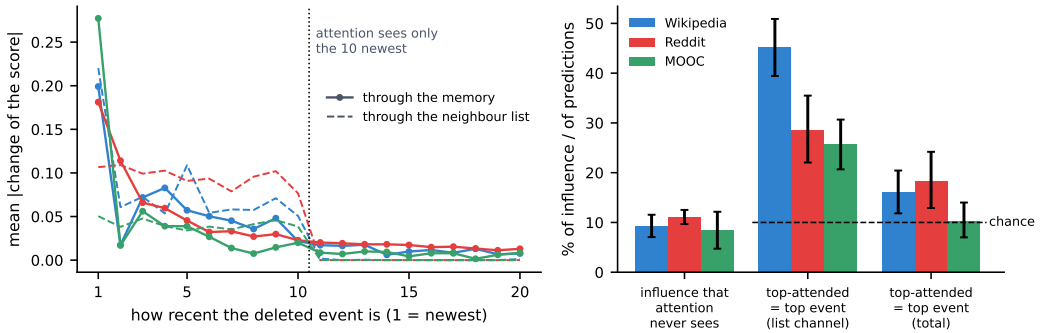


Figure 4: Left: mean absolute change of a prediction's score when one of the user's past events is deleted, by how recent the event is, through the memory and through the neighbour list. Right: share of the influence from events outside the 10 the attention layer sees, and how often the most-attended event is the most influential one (95% bootstrap intervals).

Table 6: Late events (random delays), steps of 200 events, mean ± std over seeds. MRR changes are relative to exact repair and paired within each seed; compute is in multiples of normal in-order processing. ‡Daily steps (the timestamps are days), delays with a median of 2 days, the first 200 test days with 300 queries per day; buffers of 1 and 4 days. §Real arrival times and amendments, daily steps; buffers of 1 and 4 days.

	Wikipedia	Reddit	MOOC	LastFM	tgbl-review‡	SEC§
seeds	5	5	5	3	3	5
test events that arrive late (%)	18.0	20.0	19.8	11.8	15.1	93.0
MRR of exact repair	0.640 ±0.018	0.739 ±0.019	0.152 ±0.082	0.083 ±0.021	0.359 ±0.033	0.653 ±0.022
MRR vs. exact repair (%)						
oracle (all on time)	+0.45 ±0.16	+0.13 ±0.08	+0.43 ±1.35	-0.23 ±0.41	+0.04 ±0.04	+1.74 ±0.09
arrival order	+0.13 ±0.56	-0.13 ±0.10	+0.13 ±2.84	+2.32 ±2.81	+1.37 ±4.73	+0.07 ±0.17
drop late events	+0.04 ±0.56	-0.06 ±0.06	+0.40 ±0.34	+1.77 ±2.22	+0.12 ±0.24	-0.91 ±0.30
buffer 1 h (1 day‡§)	-4.92 ±0.86	-1.53 ±0.83	-11.33 ±9.19	-4.74 ±8.69	+2.08 ±5.00	-0.06 ±0.09
buffer 4 h (4 days)	-9.49 ±0.71	-1.99 ±0.52	-14.24 ±10.94	-2.72 ±5.79	+0.24 ±2.23	-0.24 ±0.09
ε-repair, $\varepsilon = 0.05$	+0.01 ±0.05	-0.01 ±0.03	+0.32 ±0.24	+0.09 ±0.21	+0.01 ±0.05	-0.00 ±0.01
compute (× normal processing)						
full replay	16.72	69.37	48.97	5.88	34.48	54.86
exact repair (cone)	1.30	13.69	23.10	1.11	1.10	1.06
1-hop rebuild	1.27	7.18	21.92	1.10	–	1.05
ε-repair	1.24	1.89	2.20	1.08	1.07	1.04
top-1 prediction as exact repair (%)						
arrival order	78.7	91.6	69.7	79.6	82.0	84.9
ε-repair	99.3	98.4	90.7	98.3	99.4	99.8
1-hop rebuild	99.7	99.8	100.0	100.0	–	99.9

Table 7: Every delay setting: MRR change relative to exact repair (%) and compute (multiples of normal processing), mean over seeds.

	delay model	step	seeds	late %	arrival	drop	buffer 1h	buffer 4h	ε=0.05	replay	exact	ε=0.05	1-hop
Wikipedia	random	200	5	18.0	+0.13	+0.04	-4.92	-9.49	+0.01	16.72	1.30	1.24	1.27
Wikipedia	random	50	5	19.9	-0.17	-0.14	-7.92	-13.29	+0.02	37.22	1.73	1.46	1.58
Wikipedia	active users	200	5	18.9	+0.13	-0.01	-4.45	-9.29	+0.02	16.68	1.30	1.25	1.27
Wikipedia	outages	200	5	3.5	-0.02	-0.05	-5.05	-9.61	-0.00	1.07	1.01	1.01	1.01
Wikipedia	offline users	200	3	16.7	-0.07	-1.91	-3.94	-8.00	+0.01	5.79	1.07	1.06	1.06
Wikipedia	slow item shard	200	3	8.8	+0.05	-0.22	-5.16	-9.62	-0.02	13.56	1.14	1.11	1.12
Reddit	random	200	5	20.0	-0.13	-0.06	-1.53	-1.99	-0.01	69.37	13.69	1.89	7.18
Reddit	active users	200	5	12.4	-0.07	+0.01	-1.54	-1.96	+0.00	59.84	12.69	1.64	6.16
Reddit	outages	200	5	5.3	-0.01	-0.03	-1.17	-1.81	+0.00	1.13	1.04	1.04	1.04
Reddit	offline users	200	3	19.1	-0.08	-0.35	-1.47	-1.80	-0.02	24.07	2.94	1.62	2.21
Reddit	slow item shard	200	3	13.7	-0.05	-0.15	-1.36	-1.98	+0.01	60.15	9.15	1.54	3.25
MOOC	random	200	5	19.8	+0.13	+0.40	-11.33	-14.24	+0.32	48.97	23.10	2.20	21.92
MOOC	random	50	5	20.0	-2.31	+0.24	-12.78	-16.59	+0.37	118.37	77.80	2.41	58.65
MOOC	active users	200	5	4.3	+0.25	+0.28	-9.40	-13.13	+0.03	30.55	9.79	1.28	6.63
MOOC	outages	200	5	5.6	+0.20	+0.10	-9.57	-13.31	+0.03	1.14	1.09	1.07	1.09
MOOC	offline users	200	3	18.6	-0.04	+0.08	-10.46	-13.29	+0.51	17.48	7.89	1.68	4.38
MOOC	slow item shard	200	3	10.4	-0.90	-0.01	-11.87	-14.17	+0.09	39.53	15.70	1.63	12.70
LastFM	random	200	3	11.8	+2.32	+1.77	-4.74	-2.72	+0.09	5.88	1.11	1.08	1.10
LastFM	active users	200	3	6.0	-0.07	+0.73	-0.12	-3.31	+0.02	4.89	1.06	1.05	1.06
LastFM	outages	200	3	0.0	+0.02	+0.01	-0.23	-3.10	-0.00	1.00	1.00	1.00	1.00

Table 8: The likelihood-ratio attack for every attacker variant and deletion method: AUC with query access (LiRA on the link score) / with white-box access (LiRA on the cosine of the two memories), mean over seeds (5 for Wikipedia and Reddit, 3 otherwise; 16 shadow streams for tgbl-review, 64 otherwise).

	attacker	log-only	reset	restart	1-hop	ε-repair	exact
Wikipedia	full knowledge, at the request	0.84 / 0.87	0.82 / 0.68	0.82 / 0.88	0.56 / 0.54	0.58 / 0.56	0.50 / 0.50
	knows 50% of the other events	0.70 / 0.80	0.74 / 0.68	0.63 / 0.77	0.50 / 0.49	0.50 / 0.49	0.50 / 0.50
	knows 10%	0.65 / 0.75	0.67 / 0.68	0.58 / 0.70	0.51 / 0.50	0.51 / 0.50	0.50 / 0.50
	knows none of them	0.63 / 0.73	0.63 / 0.68	0.57 / 0.68	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
	20 steps after the request	0.84 / 0.87	0.81 / 0.85	0.81 / 0.85	0.55 / 0.54	0.54 / 0.56	0.50 / 0.50
	50 steps after	0.83 / 0.86	0.82 / 0.84	0.81 / 0.84	0.56 / 0.53	0.53 / 0.55	0.50 / 0.49
	$K = 100$ candidate pairs	0.92 / 0.95	0.89 / 0.85	0.92 / 0.96	0.63 / 0.60	0.64 / 0.59	0.50 / 0.50
	$K = 25$	0.97 / 0.98	0.95 / 0.95	0.98 / 1.00	0.70 / 0.69	0.73 / 0.74	0.50 / 0.51
	$K = 25$, knows 50%	0.67 / 0.80	0.78 / 0.96	0.63 / 0.84	0.47 / 0.60	0.49 / 0.49	0.50 / 0.50
	$K = 25$, knows none	0.63 / 0.72	0.65 / 0.95	0.51 / 0.68	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
Reddit	full knowledge, at the request	0.86 / 0.90	0.83 / 0.63	0.84 / 0.90	0.58 / 0.57	0.63 / 0.64	0.50 / 0.51
	knows 50% of the other events	0.65 / 0.73	0.77 / 0.63	0.56 / 0.68	0.51 / 0.51	0.52 / 0.50	0.51 / 0.50
	knows 10%	0.61 / 0.67	0.73 / 0.63	0.52 / 0.62	0.50 / 0.51	0.50 / 0.50	0.51 / 0.50
	knows none of them	0.59 / 0.65	0.73 / 0.63	0.53 / 0.62	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
	20 steps after the request	0.87 / 0.90	0.85 / 0.87	0.84 / 0.88	0.57 / 0.56	0.63 / 0.65	0.51 / 0.49
	50 steps after	0.87 / 0.90	0.86 / 0.87	0.85 / 0.87	0.56 / 0.57	0.63 / 0.65	0.51 / 0.50
	$K = 100$ candidate pairs	0.97 / 0.98	0.95 / 0.93	0.98 / 0.99	0.71 / 0.74	0.83 / 0.82	0.46 / 0.49
	$K = 25$	0.98 / 0.98	0.98 / 0.98	0.99 / 1.00	0.76 / 0.78	0.94 / 0.87	0.46 / 0.44
	$K = 25$, knows 50%	0.61 / 0.75	0.91 / 0.97	0.55 / 0.71	0.50 / 0.55	0.49 / 0.49	0.51 / 0.60
	$K = 25$, knows none	0.59 / 0.73	0.80 / 0.97	0.47 / 0.63	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
MOOC	full knowledge, at the request	0.63 / 0.65	0.75 / 0.50	0.71 / 0.71	0.51 / 0.51	0.55 / 0.55	0.51 / 0.51
	knows 50% of the other events	0.53 / 0.55	0.73 / 0.50	0.54 / 0.56	0.49 / 0.50	0.49 / 0.51	0.49 / 0.50
	knows 10%	0.50 / 0.51	0.72 / 0.50	0.53 / 0.52	0.50 / 0.49	0.49 / 0.48	0.50 / 0.49
	knows none of them	0.50 / 0.50	0.70 / 0.50	0.51 / 0.51	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
	20 steps after the request	0.65 / 0.66	0.72 / 0.74	0.69 / 0.71	0.50 / 0.50	0.54 / 0.55	0.50 / 0.50
	50 steps after	0.65 / 0.66	0.72 / 0.73	0.69 / 0.71	0.51 / 0.50	0.54 / 0.55	0.51 / 0.50
	$K = 100$ candidate pairs	0.75 / 0.78	0.97 / 0.68	0.94 / 0.96	0.52 / 0.49	0.66 / 0.68	0.52 / 0.53
	$K = 25$	0.84 / 0.85	1.00 / 0.90	0.98 / 0.99	0.63 / 0.60	0.76 / 0.71	0.46 / 0.52
LastFM	full knowledge, at the request	0.62 / 0.63	0.69 / 0.54	0.70 / 0.75	0.51 / 0.50	0.65 / 0.62	0.50 / 0.50
	knows 50% of the other events	0.51 / 0.51	0.66 / 0.54	0.53 / 0.66	0.50 / 0.50	0.51 / 0.50	0.50 / 0.50
	knows 10%	0.51 / 0.51	0.64 / 0.54	0.53 / 0.62	0.50 / 0.49	0.50 / 0.51	0.50 / 0.50
	knows none of them	0.52 / 0.50	0.63 / 0.54	0.52 / 0.58	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
	20 steps after the request	0.65 / 0.64	0.67 / 0.63	0.67 / 0.67	0.50 / 0.50	0.65 / 0.62	0.50 / 0.50
	50 steps after	0.66 / 0.63	0.68 / 0.66	0.66 / 0.64	0.51 / 0.51	0.65 / 0.63	0.50 / 0.50
	$K = 100$ candidate pairs	0.85 / 0.83	0.94 / 0.87	0.94 / 0.99	0.54 / 0.51	0.86 / 0.82	0.50 / 0.50
	$K = 25$	0.91 / 0.93	0.99 / 0.97	0.98 / 0.99	0.72 / 0.60	0.91 / 0.92	0.54 / 0.53
tgbl-review	full knowledge, at the request	0.97 / 0.98	1.00 / 0.98	0.99 / 1.00	0.66 / 0.65	0.86 / 0.81	0.50 / 0.50
	knows 50% of the other events	0.57 / 0.62	0.97 / 0.98	0.62 / 0.73	0.49 / 0.50	0.51 / 0.49	0.50 / 0.50
	knows 10%	0.53 / 0.54	0.97 / 0.98	0.55 / 0.62	0.50 / 0.50	0.49 / 0.50	0.50 / 0.50
	knows none of them	0.52 / 0.52	0.97 / 0.98	0.53 / 0.62	0.50 / 0.50	0.50 / 0.50	0.50 / 0.50
	20 steps after the request	0.97 / 0.98	1.00 / 1.00	0.99 / 1.00	0.65 / 0.65	0.86 / 0.81	0.50 / 0.50
	50 steps after	0.97 / 0.98	1.00 / 1.00	0.99 / 1.00	0.65 / 0.64	0.86 / 0.81	0.51 / 0.50

Table 9: Every deletion attack (membership AUC, mean ± std over seeds; number of seeds in brackets). Score: the model's own link score; probe: logistic regression on the two memories; LiRA: likelihood-ratio attack on the link score or on the cosine of the two memories, with counterfactual shadow streams; TPR: true-positive rate of the memory LiRA at 1% false positives; cost: updates computed by the deletion method, as a multiple of normal processing up to the request.

	method	score	probe	LiRA (score)	LiRA (memory)	TPR@1%	cost
Wikipedia (5)	log-only	0.58 ±0.02	0.64 ±0.03	0.84 ±0.01	0.87 ±0.01	22.0	0.00
	reset	0.56 ±0.02	0.88 ±0.01	0.82 ±0.01	0.68 ±0.01	11.5	0.00
	learned restart	0.52 ±0.01	0.78 ±0.01	0.82 ±0.01	0.88 ±0.01	18.5	0.05
	1-hop rebuild	0.50 ±0.01	0.51 ±0.03	0.56 ±0.02	0.54 ±0.02	2.6	0.27
	ε-repair	0.50 ±0.01	0.51 ±0.03	0.58 ±0.02	0.56 ±0.03	3.9	0.23
	exact repair	0.50 ±0.01	0.51 ±0.03	0.50 ±0.01	0.50 ±0.00	0.5	0.30
Reddit (5)	log-only	0.55 ±0.02	0.59 ±0.02	0.86 ±0.01	0.90 ±0.02	28.9	0.00
	reset	0.47 ±0.03	0.87 ±0.01	0.83 ±0.01	0.63 ±0.02	6.7	0.00
	learned restart	0.51 ±0.02	0.79 ±0.02	0.84 ±0.01	0.90 ±0.01	20.1	0.02
	1-hop rebuild	0.50 ±0.02	0.50 ±0.02	0.58 ±0.01	0.57 ±0.01	1.6	0.72
	ε-repair	0.51 ±0.02	0.50 ±0.02	0.63 ±0.01	0.64 ±0.01	6.0	0.24
	exact repair	0.50 ±0.02	0.50 ±0.02	0.50 ±0.00	0.51 ±0.01	1.0	0.79
MOOC (3)	log-only	0.51 ±0.01	0.50 ±0.02	0.63 ±0.02	0.65 ±0.01	9.7	0.00
	reset	0.68 ±0.01	0.67 ±0.01	0.75 ±0.00	0.50 ±0.00	0.4	0.00
	learned restart	0.49 ±0.03	0.67 ±0.01	0.71 ±0.02	0.71 ±0.01	7.1	0.04
	1-hop rebuild	0.51 ±0.01	0.49 ±0.01	0.51 ±0.01	0.51 ±0.01	1.0	0.95
	ε-repair	0.51 ±0.01	0.49 ±0.02	0.55 ±0.01	0.55 ±0.01	2.9	0.37
	exact repair	0.51 ±0.01	0.49 ±0.01	0.51 ±0.01	0.51 ±0.01	1.0	0.95
LastFM (3)	log-only	0.51 ±0.00	0.50 ±0.02	0.62 ±0.01	0.63 ±0.01	4.3	0.00
	reset	0.59 ±0.01	0.70 ±0.00	0.69 ±0.03	0.54 ±0.00	3.2	0.00
	learned restart	0.52 ±0.02	0.70 ±0.01	0.70 ±0.03	0.75 ±0.04	5.7	0.01
	1-hop rebuild	0.51 ±0.00	0.49 ±0.02	0.51 ±0.00	0.50 ±0.01	1.4	0.93
	ε-repair	0.51 ±0.00	0.50 ±0.01	0.65 ±0.02	0.62 ±0.01	4.2	0.12
	exact repair	0.51 ±0.00	0.49 ±0.02	0.50 ±0.00	0.50 ±0.00	0.0	0.93
tgbl-review (3)	log-only	0.52 ±0.01	0.56 ±0.00	0.97 ±0.00	0.98 ±0.00	70.5	0.00
	reset	0.02 ±0.01	1.00 ±0.00	1.00 ±0.00	0.98 ±0.00	73.2	0.00
	learned restart	0.43 ±0.03	1.00 ±0.00	0.99 ±0.00	1.00 ±0.00	96.5	0.00
	1-hop rebuild	0.51 ±0.00	0.49 ±0.01	0.66 ±0.01	0.65 ±0.01	2.6	0.18
	ε-repair	0.51 ±0.00	0.48 ±0.01	0.86 ±0.00	0.81 ±0.01	20.4	0.01
	exact repair	0.51 ±0.00	0.49 ±0.01	0.50 ±0.01	0.50 ±0.00	0.6	0.37